Aletheia — Especificação Arquitetural v0.2

Nov 9, 2026 · @ANTONIO RUFINO DA SILVA JUNIOR

Aletheia é uma linguagem em que todo termo carrega, no próprio tipo, as cinco dimensões que hoje vivem fora do código de sistemas de agentes: incerteza, autoridade, custo, localização na memória e confiabilidade da origem. O compilador `athc` verifica essas dimensões **em conjunto**, porque o problema real está na interação entre elas: um `retry` altera ao mesmo tempo a probabilidade de acerto e o orçamento; um rollback depende de quais efeitos são reversíveis; um plano gerado a partir de um log contaminado herda a contaminação.

O julgamento de tipagem central é:

```
\Gamma;\ \mathit{pc} \vdash e : \tau \ \, !\ \varepsilon \ \triangleright\ \beta \ @\ \ell
```

- **τ** — tipo do valor, incluindo refinamentos `{x:σ | φ}`, graus de certeza `σ ~ (α, δ)_D` e rótulos de integridade `σ ^ ι`.
- **ε** — linha de efeitos polimórfica; cada efeito só é executável com uma capacidade em escopo.
- **β** — orçamento graduado: vetor (tokens, USD, tempo, energia, bytes transferidos, **risco**).
- **ℓ** — localização: `host`, `dev(i)`, `ctx(agente)`, `store`.
- **pc** — integridade do contexto de controle, como em controle de fluxo de informação.

A garantia resultante: um programa Aletheia bem tipado não executa efeito sem capacidade, não excede orçamento, não deixa dado não confiável escolher ação fora de uma política auditada e só afirma uma propriedade de saída neural com o risco que um verificador ou certificado sustenta.

O que muda na v0.2

A v0.2 mantém o escopo completo e transforma os quatro nós levantados na revisão em regras do verificador e do runtime. Cada garantia passa a declarar, no próprio tipo, a hipótese de que depende.

Nó	Resolução	Seção
Composição das cinco teorias	elaboração em oito estratos com domínios de restrição isolados; cache de obrigações no Codex por hash BLAKE3	2.10

Nó	Resolução	Seção
Certificados sob adversário	grau `τ ~ (α, δ)_D`; `certify` exige fonte Trusted, ou Bounded com certificado robusto; certificado amarrado ao hash do gerador; monitor de martingales com estados tipados	2.3
Orçamento	contrato estático `Δ ⊢ e ▷ β` separado da preempção do ARX; reserva antes, liquidação depois; efeito `ContractBreach`	2.7
Efeitos remotos	chaves `IdemKey` lineares; `TimeoutUnknown` com token `Pending` linear e protocolo de reconciliação	3.5
Sagas	compensação como transição para o invariante de recuperação; `CompensationFailed` obrigatório; classes de efeito na gramática	3.4
Exemplo	Restart irrevogável fora do `atomic`; pós-condição sobre o fecho afetado; novos erros de compilação	4

1. Nome, paradigma e tese fundamental

1.1 Identidade

- **Linguagem:** Aletheia (do grego *alētheia*, "o que não está oculto"): nenhuma incerteza, efeito ou custo fica escondido do tipo.
- **Compilador:** `athc`. **Runtime:** ARX (*Aletheia Runtime eXecutive*). **Extensão:** `.ath`. **Store canônico de código:** Codex.
- **Paradigma:** funcional-primeiro, com efeitos algébricos tipados por linhas, tipos graduados quantitativos, refinamentos decididos por SMT, programação probabilística de ordem superior e atores com tipos de sessão. Mutação local via semântica de valor mutável, sem coletor de lixo.

1.2 Tese

Computação na era dos agentes tem cinco propriedades que linguagens clássicas tratam como detalhes de biblioteca:

1. **Estocástica.** A saída de um modelo é uma variável aleatória; Python a tipa como `str` e apaga a incerteza na fronteira.
2. **Com autoridade.** Uma chamada de ferramenta move dinheiro, altera infraestrutura ou envia mensagens; é efeito com consequência real, não chamada de função.

3. **Limitada em recursos.** Tokens, dólares, latência e janela de contexto são restrições duras, hoje verificadas por contadores em runtime.
4. **Heterogênea em memória.** DRAM, HBM e janela de contexto (KV-cache) têm unidades, custos de transferência e políticas de despejo distintos.
5. **Adversarial.** Todo texto lido do mundo pode conter instruções: injeção de prompt é o novo estouro de buffer.

Não faltam ferramentas para cada dimensão isolada. Falta uma que raciocine sobre a **interação** entre elas:

- `retry 3` multiplica o custo por 3 e muda a probabilidade de obter resposta válida — orçamento e probabilidade no mesmo termo.
- Um LLM que leu um log contaminado produz um plano; quem pode executá-lo depende do fluxo de dados e da capacidade — integridade e autoridade no mesmo termo.
- Rollback só existe se todos os efeitos do bloco forem compensáveis — efeitos e transações no mesmo termo.
- Compactar contexto para caber na janela transforma fato exato em resumo — memória e certeza no mesmo termo.

Combinações atuais cobrem colunas isoladas; nenhuma tem um único verificador que veja a linha inteira:

Sistema	Distribuições nativas	Efeitos com capacidade	Integridade anti-injeção	Orçamento no tipo	Formas de tensor no tipo	Verificação formal	Memória sem GC
Rust	—	—	—	—	parcial (const generics)	externa (Kani, Prusti, Verus)	sim
Mojo	—	—	—	—	parcial	—	sim
Python + Pydantic / DSPy	—	—	—	—	—	—	—
Koka / OCaml 5	—	sim (Koka)	—	—	—	—	parcial
Pyro / Gen / Stan	sim	—	—	—	parcial	—	—
Dafny / F* / Lean	—	parcial (F*)	—	—	—	sim	—

Sistema	Distribuições nativas	Efeitos com capacidade	Integridade anti-injeção	Orçamento no tipo	Formas de tensor no tipo	Verificação formal	Memória sem GC
CaMeL (padrão de sistema, 2025)	—	sim	sim	—	—	—	—
Aletheia	**sim**	**sim**	**sim**	**sim**	**sim**	**sim, em camadas**	**sim**

1.3 Formato de representação: uma fonte, três projeções

O programa não é um arquivo de texto. É um grafo tipado e endereçado por conteúdo, com três projeções sem perda entre si.

- **Codex — a fonte da verdade.** Cada definição é uma AST tipada normalizada, identificada pelo hash BLAKE3 da sua forma; o conjunto forma um DAG de Merkle. Nomes são metadados: renomear não muda hashes, não invalida builds e não exige reprovar nada. Provas e certificados ficam associados ao hash e são verificados uma única vez.
- **Superfície textual (`.ath`) — para humanos.** Formatação canônica única, sem variação de estilo, com round-trip exato para o Codex. Palavras-chave ASCII curtas; `→` e `⩽` são apenas renderização opcional de `->` e `<=`.
- **ACS (*Agent Context Slice*) — para agentes.** Dado um alvo de edição, `athc` calcula o fecho mínimo de interfaces de que ele depende: assinaturas, contratos, capacidades e orçamentos, sem corpos. O alvo aparece como **buraco tipado**; o agente devolve um `patch` (substituição de subárvore), verificado antes do merge. Patch mal tipado nunca entra no Codex.

Exemplo de fatia ACS entregue a um agente que vai reescrever um handler:

```
§slice ops.remediate/Remediator.alert
#a91f3c
use planner         : Model[ctx=128k, tok=o200k]                        #3c0e
use SafeRemediation : policy ClusterState -> Diagnosis -> Bool          #77d2
use apply           : (Cap[Cluster.Write], Fix ^ Endorsed, own IdemKey) ->
Receipt
                        ! {Cluster.Write[fp]} compensable queryable
#c1b4
use restart         : (Cap[Cluster.Write], Kick ^ Endorsed, own IdemKey) ->
Receipt
                        ! {Cluster.Write[fp]} irrevocable idempotent
#5e02
budget  tok<=60k usd<=0.50 wall<=120s risk<=(0,0)   under Δ#k8s
hole ?h1 : Outcome ! {Infer, Cluster, Page, TimeoutUnknown,
                      CompensationFailed, ContractBreach}   -- alvo da edição
```

Decisões de projeto voltadas a geradores neurais:

- **Uma forma canônica por programa** reduz a entropia do que o modelo precisa emitir.
- **Nomes locais preservados**, não índices de De Bruijn: modelos raciocinam melhor com nomes. Dependências externas aparecem como alias curto mais hash.
- **Edição estrutural:** o agente nunca emite texto que precise ser reparseado num contexto ambíguo.
- **O compilador é um servidor MCP:** agentes pedem fatias, submetem patches e recebem diagnósticos estruturados pelo mesmo protocolo que usam para qualquer outra ferramenta.

2. Sistema de tipos e primitivas nativas

O núcleo formal λ_Ath funde cinco teorias já estabelecidas — efeitos por linhas, tipos quantitativos, refinamentos, mônadas graduadas e fluxo de informação — num único julgamento. As primitivas abaixo são as formas concretas dessa fusão.

2.1 Gramática de tipos do núcleo

```
τ, σ ::= B                          base: Nat, Int, F32, Bool, Text[<= n tok]
       | τ ->⟨ε, β⟩ σ               função com efeitos ε e custo β
       | {x : τ | φ}                refinamento; φ em fragmento decidível
       | Dist τ                     distribuição sobre τ
       | τ ~ (α, δ)_D               grau estocástico calibrado em D (2.3)
       | τ ^ ι                      integridade (2.3, 2.6)
       | Tensor[d, S] @ ℓ           dtype d, forma S com eixos nomeados, local ℓ
       | Cap[E]                     capacidade linear para os efeitos E
       | Model[τ => σ, κ]           modelo neural com configuração κ
       | Cert[φ, α, δ, D, h]        certificado conformal; h = hash do gerador
       | Seg[n, π]                  segmento de contexto: n tokens, proveniência
π
       | Claim τ | Obs τ            afirmação gerada vs observação de ferramenta
       | IdemKey | Pending[k]       chave de idempotência; resultado remoto em
aberto
       | Chan[P]                    canal com protocolo de sessão P
ι    ::= Untrusted | Bounded(ρ, ε, x₀) | Endorsed(P) | Trusted
q    ::= 0 | 1 | ω                  multiplicidade: apagado, linear, irrestrito
```

As multiplicidades seguem a teoria de tipos quantitativa. Capacidades, buffers de acelerador, regiões de contexto, chaves IdemKey e tokens Pending são lineares (q = 1); tipos, provas e certificados são apagados em runtime (q = 0).

2.2 Distribuições como valores

`Dist τ` é uma mônada de probabilidade com semântica em espaços quase-Borel, o que admite distribuições sobre funções sem perder solidez.

- `x <~ d` amostra; `observe(d, φ)` condiciona; `infer[m](prog)` aplica um motor de inferência.
- Motores de inferência são **handlers de efeito**: enumeração exata, amostragem por importância, SMC e HMC/NUTS executam o mesmo modelo sem reescrevê-lo.
- Todo sorteio é um efeito registrado no journal do ARX; a reexecução é bit a bit.

2.3 Graus de certeza, risco e certificados sob adversário

Princípio central: **sempre que existir verificador decidível, a segurança probabilística vira segurança determinística mais vivacidade probabilística.** Verificadores decidíveis são robustos a adversários porque checam a saída contra estado confiável. Certificados estatísticos valem apenas sob a distribuição de calibração, e o sistema de tipos codifica exatamente essa diferença.

Grau estocástico. O tipo `τ ~ (α, δ)_D` afirma: com probabilidade de pelo menos 1 − δ sobre o sorteio do conjunto de calibração D, o risco de aceitar um valor que viole φ é no máximo α, para entradas permutáveis com D.

```
\Pr_{D}\Big[\ \Pr_{x}\big(\mathsf{aceito}(x) \wedge \neg\varphi(x)\big) \le
\alpha\ \Big] \ \ge\ 1 - \delta
```

- O limiar é calibrado por controle de risco condicional ao conjunto de calibração (RCPS, Learn-then-Test), não pela versão apenas marginal.
- α é erro marginal: uma média sobre a distribuição de entradas, nunca uma probabilidade sobre uma saída individual.
- φ é definida pelos rótulos de D; a garantia vale para esses rótulos, e o hash de D registra quem os produziu.

Proveniência	Como surge	Grau (α, δ)	Hipótese registrada no Codex
`proved`	prova em E4–E5 ou T2	(0, 0)	nenhuma
`checked`	`refine … by φ`, φ decidível	(0, 0); falha vai para `else`	nenhuma
`conformal`	`certify … by c`, fonte Trusted	(α, δ)	permutabilidade entre D e produção; hash do gerador
`robust`	`certify` com certificado robusto, fonte Bounded	(α, δ) sob perturbação ⩽ ε	modelo de ameaça (ρ, ε) e âncora confiável x_0
`assumed`	`assume` explícito	declarado	marcada na auditoria

```
\dfrac{\Gamma \vdash d : \mathsf{Dist}\ \sigma \triangleright \beta_d \qquad
\Gamma \vdash \varphi : \sigma \to \mathsf{Bool}\ \text{pura e decidível}
\qquad k \in \mathbb{N}}{\Gamma \vdash \mathsf{refine}\ d\ \mathsf{by}\
\varphi\ \mathsf{retry}\ k : \mathsf{Option}\ \{x{:}\sigma \mid \varphi\, x\}
\triangleright k \cdot \beta_d}\ \ \text{(T-Refine)}
```

Composição. Pela desigualdade de Boole, α soma a cada uso. δ soma uma única vez por certificado distinto, porque o evento "D foi uma amostra ruim" é o mesmo em todas as chamadas que usam o mesmo certificado. O risco segue como componente do orçamento, agora como par: `risk <= (0.05, 1e-3)`.

```
\mathit{risk}(e) \;=\; \Big(\ \sum_{u \,\in\, \mathsf{usos}(e)}
\alpha_{c(u)},\ \ \sum_{c \,\in\, \mathsf{certs}(e)} \delta_c\ \Big)
```

Fonte não confiável. Um adversário escolhe entradas fora da distribuição de calibração, e a permutabilidade deixa de valer. Por isso a regra exige que a integridade das entradas do gerador, ι_src(v), seja Trusted. Endorsed não basta: o endosso limita o que o adversário escolhe, mas não torna a escolha permutável.

$$\dfrac{\Gamma \vdash v : \sigma \qquad c : \mathsf{Cert}[\varphi,\ \alpha,\ \delta,\ D,\ h] \qquad h = \mathsf{hash}(\mathsf{gen}(v)) \qquad \iota_{\mathit{src}}(v) = \mathsf{Trusted}}{\Gamma \vdash \mathsf{certify}\ v\ \mathsf{by}\ c : \mathsf{Option}\ (\{x{:}\sigma \mid \varphi\, x\} \sim (\alpha, \delta)_D)\ !\ \{\mathsf{Cert}\ c\} \triangleright \langle \mathit{risk} = (\alpha, \delta) \rangle}\ \ \text{(T-Certify)}$$

Para entradas que são perturbação limitada de uma âncora confiável, o reticulado ganha `Bounded(ρ, ε, x₀)`. Esse rótulo só é obtido por um teste decidível de distância contra x_0 — por exemplo, distância de edição ⩽ 8 sobre um template confiável. O certificado robusto é calibrado com o pior escore na bola de raio ε′, limitado por um verificador de redes neurais ou por suavização aleatória.

$$\dfrac{\Gamma \vdash v : \sigma \qquad c : \mathsf{RobustCert}[\varphi,\ \alpha,\ \delta,\ D,\ h,\ \rho,\ \varepsilon'] \qquad h = \mathsf{hash}(\mathsf{gen}(v)) \qquad \iota_{\mathit{src}}(v) = \mathsf{Bounded}(\rho, \varepsilon, x_0) \qquad \varepsilon \le \varepsilon'}{\Gamma \vdash \mathsf{certify}\ v\ \mathsf{by}\ c : \mathsf{Option}\ (\{x{:}\sigma \mid \varphi\, x\} \sim (\alpha, \delta)_D)\ !\ \{\mathsf{Cert}\ c\} \triangleright \langle \mathit{risk} = (\alpha, \delta) \rangle}\ \ \text{(T-Certify-Robust)}$$

Qualquer outra fonte é o erro E0421. Quando não existe (ρ, ε) com significado — texto arbitrário, como na injeção de prompt — o único caminho até um tipo refinado é `refine … by policy`.

Amarração ao gerador. O índice h de `Cert` é o hash BLAKE3 do fecho gerador: pesos, tokenizer, parâmetros de decodificação, template de prompt, pré-processamento e escore. Hashes são literais no tipo, e a igualdade é checada no estrato E0. Novo modelo, novo template ou novo escore tornam o certificado antigo tipo-incompatível (E0430), em compilação.

Monitor de deriva. Cada certificado tem um estado no ARX, exposto ao tipo pelo efeito `Cert c`. Duas martingales correm em paralelo: uma sobre escores sem rótulo, que detecta mudança na distribuição dos escores, e outra sobre perdas em amostras auditadas por humanos, que detecta mudança nos rótulos. Com p-valores conformais p_t, usa-se a martingale de potência, e a desigualdade de Ville limita o falso alarme:

$$M_t = \prod_{i=1}^{t} \kappa\, p_i^{\,\kappa - 1},\ \ \kappa \in (0,1) \qquad \Pr\Big[\ \sup_{t \ge 1} M_t \ge \tfrac{1}{\lambda}\ \Big] \le \lambda\ \ \text{sob permutabilidade}$$

Estado	`certify` devolve	Próximo estado	Gatilho
Valid	`Some v` se s(v) ⩽ λ̂, senão `None`	Suspect	M_t ⩾ 1/λ_w
Suspect	como Valid; cada aceitação vai para auditoria prioritária	Revoked	M_t ⩾ 1/λ em qualquer das duas martingales
Valid ou Suspect	—	AuditDue	k aceitações desde a última auditoria rotulada
AuditDue	`None`	Valid ou Revoked	resultado da auditoria
Revoked	`None`; emite o efeito `CertRevoked(c)`	Valid, com novo certificado c′	`athc certify` com novo D′

$$\dfrac{\sigma(c) \in \{\mathsf{Valid}, \mathsf{Suspect}\} \qquad s(v) \le \hat\lambda_c}{\langle \mathsf{certify}\ v\ \mathsf{by}\ c,\ \sigma \rangle \longrightarrow \langle \mathsf{Some}\ v,\ \sigma[n_c \mapsto n_c + 1] \rangle} \qquad \dfrac{\sigma(c) \in \{\mathsf{AuditDue}, \mathsf{Revoked}\}}{\langle \mathsf{certify}\ v\ \mathsf{by}\ c,\ \sigma \rangle \longrightarrow \langle \mathsf{None},\ \sigma \rangle}$$

A garantia, enunciada com precisão: sob permutabilidade, o monitor custa apenas vivacidade, com falso alarme ⩽ λ. Sob deriva, (α, δ) não vale entre o início da deriva e o alarme. O ARX não promete detectar toda deriva; promete que no máximo k saídas são aceitas entre auditorias rotuladas e que, depois de qualquer alarme, nenhuma saída é aceita sem certificado válido.

2.4 Tensores com forma dependente

Eixos são nomeados; tamanhos são naturais no nível de tipo; restrições ficam na aritmética de Presburger, que é decidível. Trocar eixos, broadcast incompatível e contração no eixo errado são erros de compilação.

```
fn attention[B, T, H, D](
  q: &Tensor[bf16, {b: B, t: T, h: H, d: D}] @dev,
  k: &Tensor[bf16, {b: B, s: T, h: H, d: D}] @dev,
  v: &Tensor[bf16, {b: B, s: T, h: H, d: D}] @dev,
) -> own Tensor[bf16, {b: B, t: T, h: H, d: D}] @dev
  where D <= 256, T % 128 == 0
{
  let w = softmax(einsum("b t h d, b s h d -> b h t s", q, k) / sqrt(D), axis
= s)
  einsum("b h t s, b s h d -> b t h d", w, v)
}
```

A cláusula `T % 128 == 0` não é só verificação: ela permite ao backend escolher um kernel ladrilhado sem ramificação em runtime.

2.5 Modelos e nós de inferência

`model` declara um valor do tipo `Model[τ => σ, κ]`. A configuração κ inclui hash dos pesos, tokenizer, janela, parâmetros de decodificação, preço e latência — o modelo é parte da identidade do programa, não uma URL.

- **Decodificação restrita derivada do tipo.** A gramática de σ (ADTs, refinamentos regulares, `Text[<= n tok]`) compila para um autômato sobre tokens; a saída é sintaticamente válida por construção.
- **Teto de saída derivado do tipo.** `Text[<= 400 tok]` mais uma lista de 1 a 3 ações fixa `max_out`, que entra no orçamento estático.
- **Contaminação atravessa o modelo.** A saída é `Claim` e herda a integridade da entrada.

```
\dfrac{\Gamma \vdash m : \mathsf{Model}[\tau \Rightarrow \sigma,\ \kappa]
\qquad \Gamma;\mathit{pc} \vdash e : \tau^{\iota} \triangleright \beta_e
\qquad |e|_{\kappa} + \mathit{out}(\sigma) \le \kappa.\mathit{ctx}}
{\Gamma;\mathit{pc} \vdash \mathsf{infer}\ m\ e : (\mathsf{Dist}\
(\mathsf{Claim}\ \sigma))^{\iota \sqcap \mathit{pc}}\ !\ \{\mathsf{Infer}\
m\} \triangleright \beta_e \oplus \mathit{cost}_{\kappa}(e, \sigma)}\ \
\text{(T-Infer)}
```

Inferência de tipos assistida por modelo segue a mesma regra de autoridade: `athc suggest` usa um modelo para propor invariantes de laço, refinamentos, anotações de buracos e esboços de prova. O modelo propõe; o kernel decide. Nenhuma sugestão entra no Codex sem passar por T0–T2.

2.6 Capacidades e integridade

- **Capacidades** são lineares, não forjáveis, atenuáveis (`pay.attenuate(max_usd = 100)`) e revogáveis pelo ARX. Efeito sem capacidade em escopo é inexprimível.
- **Integridade** forma um reticulado em diamante: Untrusted ⊑ Endorsed(P) ⊑ Trusted e Untrusted ⊑ Bounded(ρ, ε, x_0) ⊑ Trusted, com Endorsed e Bounded incomparáveis (2.3). Cada ferramenta declara a integridade mínima de seus argumentos de controle e a integridade de sua saída.
- **Fluxos implícitos:** ramificar sobre dado Untrusted rebaixa o pc, e todo efeito no ramo herda o rebaixamento. "Se o log disser X, apague Y" é erro de tipo.
- **Disparo não é controle:** um handler acionado por evento externo não rebaixa o pc. O atacante pode fazer o agente agir, limitado pelo orçamento; nunca escolher a ação.

```
\dfrac{\Gamma \vdash c : \mathsf{Cap}[E] \qquad t : \mathsf{Tool}[A \to B,\
E,\ \iota_{\min},\ \iota_{\mathit{out}}] \qquad \Gamma;\mathit{pc} \vdash a :
A^{\iota_a} \qquad \iota_{\min} \sqsubseteq \iota_a \sqcap \mathit{pc}}
{\Gamma;\mathit{pc} \vdash t(c, a) : (\mathsf{Obs}\
B)^{\iota_{\mathit{out}}}\ !\ E}\ \ \text{(T-Call)}
```

```
\dfrac{\Gamma \vdash v : \sigma^{\iota} \qquad P : \mathsf{policy}\ \sigma\
\text{pura, total, decidível e Trusted}}{\Gamma \vdash \mathsf{refine}\ v\
\mathsf{by}\ P : \mathsf{Option}\ (\{x{:}\sigma \mid P\,
x\})^{\mathsf{Endorsed}(P)}}\ \ \text{(T-Endorse)}
```

Endosso robusto: um adversário que controla todo o texto Untrusted escolhe, no máximo, um elemento de {x | P x}. O raio de explosão de uma injeção de prompt é o conjunto aceito pela política, não o conjunto do que o modelo consegue gerar. O padrão CaMeL aplica essa ideia num interpretador em runtime; Aletheia a move para a tipagem, e a violação vira erro de compilação.

2.7 Orçamento: contrato estático e preempção distribuída

O orçamento tem duas garantias de natureza diferente, com julgamentos diferentes. O compilador prova um contrato; o ARX impõe um teto. Nenhuma das duas trata a rede como determinística.

Contrato estático. Δ é o ambiente de contratos: para cada dependência, latência, preço e teto de saída declarados. O compilador prova `Δ ⊢ e ▷ β`, com β = (tok, usd, wall, joule, bytes, risk). Em sequência, todas as componentes somam; em paralelo, todas somam exceto o tempo, que toma o máximo.

```
\dfrac{\Delta;\Gamma \vdash e_1 : \tau_1 \triangleright \beta_1 \qquad
\Delta;\Gamma, x{:}\tau_1 \vdash e_2 : \tau_2 \triangleright \beta_2}
{\Delta;\Gamma \vdash \mathsf{let}\ x = e_1;\ e_2 : \tau_2 \triangleright
\beta_1 \oplus \beta_2} \qquad \dfrac{\Delta;\Gamma \vdash e_1 : \tau_1
\triangleright \beta_1 \qquad \Delta;\Gamma \vdash e_2 : \tau_2
\triangleright \beta_2}{\Delta;\Gamma \vdash \mathsf{par}\{e_1;\ e_2\} :
\tau_1 \times \tau_2 \triangleright \beta_1 \otimes \beta_2}
```

```
\Delta \vdash e \triangleright \beta \ \ \wedge\ \ \mathsf{honra}
(\text{traço}, \Delta) \ \implies\ \mathsf{custo}(\text{traço}) \preceq \beta
```

O ARX mede cada chamada contra Δ. Qualquer violação dispara o efeito algébrico `ContractBreach(d, observado, declarado)`, de tratamento obrigatório. Toda execução, portanto, ou custa no máximo β, ou sinalizou, no instante da violação, qual contrato quebrou.

Preempção distribuída. O teto duro B é imposto pelo ARX independentemente dos contratos:

- **Reserva antes, liquidação depois.** Antes de cada chamada c, o ARX reserva r_c, o custo máximo de c sob Δ. Se o saldo não cobre r_c, `BudgetExhausted` é levantado antes de a chamada começar. Depois, a reserva é liquidada pelo uso faturado. Invariante: reservado + liquidado ⩽ B em todo instante.
- **Prazos absolutos.** A concorrência estruturada propaga prazos: o de uma tarefa filha é o mínimo entre o seu e o do pai. O compilador insere pontos de cancelamento em arestas de retorno de laços e em sítios de efeito. Kernels de acelerador são preemptados entre kernels, e cada um tem custo estático derivado das formas (E4).
- **O que a preempção não faz.** Ela não para uma operação remota já enviada. Prazo expirado em efeito remoto produz resultado desconhecido, não falha (3.5).
- **Obrigações sobrevivem ao orçamento; trabalho novo, não.** Esgotado B, o ARX bloqueia efeitos novos e transfere cada token `Pending` ao reconciliador durável, que tem orçamento próprio declarado na implantação.

Componente	Contrato estático	Imposição no ARX	Resíduo tratado
tokens de entrada	exato: tokenizer fixo, prompt é AST	—	nenhum
tokens de saída	teto derivado do tipo	corte local do stream em `max_out`	nenhum

Componente	Contrato estático	Imposição no ARX	Resíduo tratado
USD	preço de Δ × tokens	reserva e liquidação; fatura conferida contra Δ	`ContractBreach` se o preço divergir
tempo	soma das latências de Δ no pior caminho	prazos absolutos e cancelamento	`TimeoutUnknown` em efeitos remotos
risco	soma de (α, δ), ver 2.3	estados do monitor de certificados	deriva não detectada limitada a k saídas

Para laços estocásticos vale ainda a análise de custo esperado: amostrar até validar, com taxa de sucesso q, custa β/q em média.

2.8 Contexto, afirmação e observação

- `Text[<= n tok]` amarra a contagem ao tokenizer do modelo de destino. Como prompts são ASTs, o tamanho máximo de cada prompt é calculável; estouro de janela é erro de compilação, nunca truncamento silencioso.
- `Seg[n, π]` registra a proveniência π ∈ {Exact(h), Summary(hs, m), Retrieved(q, s)}. Compactação é declaradamente com perda, e fatos extraídos de um `Summary` são `Claim`.
- `infer` produz `Claim`; ferramentas produzem `Obs`. `verify c using t` converte afirmação em observação quando a ferramenta a confirma.
- Política padrão: precondições de efeitos irrevogáveis exigem `Obs` ou prova.

2.9 Modelo de prova em camadas

Camada	Garante	Mecanismo	Decidível	Custo típico
T0 Tipos	forma, efeitos, capacidades, integridade, localização	inferência bidirecional, linhas de efeito	sim	milissegundos
T1 Refinamentos	invariantes aritméticas, formas de tensor, contexto, orçamento estático	SMT (Z3, cvc5) em QF_LIA, Presburger e funções não interpretadas	sim, no fragmento	segundos

Camada	Garante	Mecanismo	Decidível	Custo típico
T2 Provas	correção funcional de módulos críticos	blocos `proof` checados por kernel pequeno estilo LCF; exportação para Lean 4	não (interativo)	horas, humanas ou de agente
TS Estatística	propriedades semânticas de saídas neurais	certificados conformais com hash do conjunto de calibração	não se aplica	uma calibração por versão de modelo

Nenhuma camada aceita "talvez": timeout de SMT é erro, não aprovação. Provas e certificados ficam no Codex, indexados pelo hash do que garantem.

2.10 Elaboração estratificada e cache de obrigações

As cinco teorias convivem no verificador porque nunca resolvem restrições juntas. O sistema é gerador de restrições no estilo HM(X), parametrizado por um produto de domínios. Cada restrição nasce em exatamente um domínio, e os domínios são resolvidos em estratos, cada um por um solver de teoria pura.

Estrato	Domínio de restrição	Solver	Decidível	Consome
E0 Estrutura	tipos simples, ADTs, polimorfismo, hashes literais	elaboração bidirecional e unificação	sim	—
E1 Multiplicidade	semianel {0, 1, ω}	propagação	sim	E0
E2 Efeitos	linhas de efeito com footprints simbólicos	unificação de linhas	sim	E0
E3 Integridade	reticulado finito e pc	ponto fixo em semirreticulado finito	sim, tempo linear	E0, E2

Estrato	Domínio de restrição	Solver	Decidível	Consome
E4 Índices e formas	Presburger com divisibilidade; monômios como átomos	Z3 / cvc5 em QF_LIA	sim	E0
E5 Refinamentos	cláusulas de Horn com qualificadores sobre UF, LIA e datatypes	CHC com abstração de predicados (liquid)	sim, com qualificadores finitos	E0, E4
E6 Orçamento	polinômios monótonos sobre índices	substituição pelos limites superiores; tokens em QF_LIA; USD, tempo e energia em QF_LRA	sim	E2, E4, E5
E7 Risco	pares (α, δ); δ uma vez por certificado	aritmética racional ground	sim	E2, E5

As camadas de 2.9 se relacionam assim: T0 corresponde a E0–E3 e T1 a E4–E7; T2 e TS seguem como antes.

Isolamento de domínios. Três regras garantem que nenhum solver veja símbolos de outro:

1. **Separação índice–valor** (estilo Dependent ML). Tipos mencionam termos de uma linguagem de índices — naturais, conjuntos de efeitos, rótulos, pares de risco — e nunca valores de programa. Não há computação de programa dentro de tipos, e cada domínio tem assinatura própria.
2. **Reflexão sintática.** `index(n)` só aceita n cujo refinamento esteja sintaticamente em Presburger. A restrição é copiada para E4 durante E0, sem esperar que E5 resolva nada; por isso E5 → E4 não é aresta do grafo de estratos.
3. **Fluxo só para frente.** Um estrato consome apenas soluções ground ou índices de estratos anteriores. Como nenhuma restrição mistura assinaturas, nenhuma combinação Nelson–Oppen é necessária.

Decidibilidade de E4 e E6. Produtos de tamanhos simbólicos, como o reshape para B·T, entram como monômios normalizados tratados como átomos; isso decide igualdade de formas sem aritmética não linear. Custos são polinômios de coeficientes não negativos sobre índices, logo monótonos: o pior caso está nos limites superiores. E6 substitui esses limites, provados em E4 e E5, e reduz a checagem a aritmética ground. Índice sem limite superior é erro de orçamento, não um "talvez".

Teorema-alvo T8. A elaboração estratificada é correta e completa em relação ao sistema declarativo para programas cujas restrições caem nos fragmentos declarados. E0–E3 têm soluções principais; E4–E7 são de checagem. Erros são relatados no primeiro estrato que falha, com o nome do domínio, o que mantém as mensagens legíveis.

Cache de obrigações no Codex. Obrigações e resumos de definições são indexados por hash:

```
\mathsf{key}(o) = \mathrm{BLAKE3}\big(\mathit{dom}_i \,\|\, \mathsf{nf}(C)
\,\|\, \mathit{cfg}_i\big) \qquad \mathsf{key}_i(f) =
\mathrm{BLAKE3}\big(\mathsf{hash}(f) \,\|\, \{\mathsf{sum}_j(g) : g \in
\mathsf{deps}(f),\ j \preceq i\}\big)
```

- **Forma normal.** Uma obrigação é a tupla (domínio, restrição normalizada, configuração da teoria, versão do solver). A normalização renomeia variáveis pela ordem da primeira ocorrência, ordena conjunções pelo hash, dobra constantes e remove conjunções triviais; obrigações equivalentes por essas regras colidem na mesma chave.
- **Entradas.** `Valid(certificado)`, `Invalid(contraexemplo)` ou `Unknown(esforço gasto)`. `Unknown` nunca vira `Valid`; ele registra o esforço para não repetir a mesma tentativa.
- **Certificados independentes.** Provas no formato Alethe, emitidas pelo cvc5 e checadas por um verificador independente da classe do Carcara. O solver sai da base confiável: o cache pode ser compartilhado entre máquinas e agentes, e envenená-lo exige forjar uma prova que o verificador aceite.
- **Resumos por estrato.** Cada definição tem, por estrato, tipo principal, linha de efeitos, assinatura de integridade, polinômio de custo e par de risco, indexados pelo hash da definição e pelos resumos das dependências nos estratos que consome.
- **Corte antecipado.** Mudar um corpo que preserva o resumo não reelabora dependentes. Mudar uma declaração de orçamento invalida apenas E6 e E7 das definições afetadas; E0–E5 vêm do cache. É isso que impede recomputar a interação entre efeitos, orçamento e integridade.
- **Rebaixamento.** As formas resolvidas em E4 viram dimensões estáticas ou simbólicas nos tipos `tensor` do MLIR, e os fatos de divisibilidade viram atributos que liberam ladrilhamento sem guardas em runtime.

3. Memória, concorrência e runtime ARX

O ARX trata DRAM, HBM, janela de contexto, armazenamento durável e o mundo externo como cinco localizações de um único modelo de memória. Cada uma tem sua unidade, sua disciplina de posse e um custo de transferência que entra no orçamento.

3.1 Hierarquia de localizações

Localização	Meio	Unidade	Disciplina	Transferência entra no orçamento como
`@host`	DRAM	bytes	valor mutável, sem GC	—
`@dev(i)`	HBM / VRAM	bytes	buffers lineares, liberação determinística	bytes ÷ banda (PCIe, NVLink)
`@ctx(a)`	janela de contexto + KV-cache	tokens	região linear por agente, compactação tipada	tokens de prefill
`@store`	disco / objeto	bytes	durável, endereçado por conteúdo	latência de E/S
`@world`	sistemas externos	efeitos	capacidades + transações	custo declarado da ferramenta

3.2 Gestão de memória

- **Host — semântica de valor mutável.** Valores não armazenam referências; parâmetros são `let` (leitura), `inout` (mutação exclusiva) ou `sink` (consumo). O resultado é segurança de memória sem GC e sem anotações de lifetime, uma fonte frequente de falhas em código Rust gerado por modelos.
- **Acelerador — linearidade.** `own Tensor @dev` é consumido exatamente uma vez; o compilador insere a liberação no último uso. Não há vazamento de HBM nem uso após liberação entre streams. `move x to dev(0)` é explícito e custa bytes no orçamento.
- **Inferência de localização.** Valores sem anotação recebem local por fluxo de custo mínimo sobre o grafo de dados; o IDE mostra a decisão e o custo de cada transferência.
- **Estado de agente com ciclos.** Arenas por época, liberadas ao fim de cada evento. Estado de longa duração vive em `@store` como estrutura persistente (HAMT).
- **Contexto — memória virtual de tokens.** O ARX mantém o KV-cache como árvore radix de prefixos. Como prompts são ASTs tipadas, o compilador separa o estático do dinâmico e emite um **plano de prefixos**: sistema, esquemas de ferramentas e políticas primeiro, maximizando reuso entre chamadas e entre agentes.
- **Despejo tipado.** Políticas de despejo são funções declaradas — `drop`, `summarize(m, keep = fatos)`, `page_out(store, index)` — e as que perdem informação alteram a proveniência do segmento.

3.3 Concorrência estruturada e atores

- `par`, `race` e `scope`: nenhuma tarefa filha sobrevive ao escopo, cancelamento se propaga e o orçamento das filhas é subalocado do pai (`par[split = 0.7 / 0.3]`).
- **Agentes são atores**: caixa de mensagens tipada, capacidades próprias, orçamento próprio, nenhum estado mutável compartilhado implícito.
- **Tipos de sessão multipartes** regem protocolos entre agentes. O protocolo global é projetado em tipos locais, e cada agente é verificado contra sua projeção. Implementações bem tipadas de protocolos bem formados são livres de deadlock e de mensagens fora de ordem.

```
protocol CodeReview(Author, Reviewer, CI) {
  Author   -> Reviewer : Patch
  Reviewer -> CI       : Patch
  CI       -> Reviewer : Report
  choice at Reviewer {
    Approve: Reviewer -> Author : Merge
    Reject:  Reviewer -> Author : Reasons; continue
  }
}
```

- **Consistência no tipo.** `Shared[T, crdt]` converge sem coordenação e só aceita operações comutativas. `Shared[T, linear]` usa log replicado com consenso Raft e é linearizável. Operação não comutativa sobre `crdt` é erro de tipo.

3.4 Transações sobre o mundo: semântica exata de sagas

Compensar não é desfazer o passado. É executar uma nova transição, provada para levar o mundo a um estado que satisfaz um invariante de recuperação declarado.

Estado abstrato. Cada módulo de ferramentas declara uma assinatura abstrata W do mundo — por exemplo `W.spec(d)` e `W.ready(d)` para um deployment d. Cada efeito declara um contrato sobre W, descarregado em E5 sob a hipótese de que a ferramenta o honra, e confirmado em runtime por sondas.

Classes de efeito na gramática.

```
class    ::= "reversible"
           | "compensable" "by" expr "restoring" pred ("commutes" "on"
footprint)?
           | "irrevocable" ("observed" "by" pred)?
provider ::= "provider" ("idempotent" | "queryable" | "opaque")
```

Classe	Exemplo	Dentro de `atomic`	Em falha
`reversible`	estado interno (STM)	permitido	desfeito pelo runtime
`compensable`	escalar réplicas, reverter revisão	permitido	C leva a um estado que satisfaz R; falha vira `CompensationFailed`
`irrevocable`	reiniciar pod, enviar mensagem, pagar	proibido — só via `commit`	a sonda confirma ou o agente escala; nunca compensa

Semântica da compensação. Se A leva S_1 a S_2, a compensação declarada C leva S_2 a S_1', e S_1' satisfaz o invariante de recuperação — não precisa ser S_1. Como um efeito remoto pode terminar em resultado desconhecido (3.5), C precisa ser total sobre "aconteceu" e "não aconteceu": sua precondição é P ∨ Q.

```
\{P\}\ A\ \{Q\} \qquad \{P \vee Q\}\ C\ \{R\} \qquad \textstyle\bigwedge_{i \in \mathsf{exec}} R_i \implies \mathit{Inv}
```

O bloco `atomic … recover Inv` declara o invariante, e E5 descarrega a obrigação de que a conjunção dos R_i dos efeitos executados implica Inv.

Ordem e paralelismo. Efeitos que declaram `commutes on F` com footprints disjuntos têm as compensações executadas em paralelo; a regra de enquadramento da lógica de separação garante que uma compensação não destrói o R de outra. Com footprints sobrepostos, as compensações rodam em ordem reversa estrita, e a obrigação passa a ser a composição sequencial por precondição mais fraca.

```
\dfrac{\{P\}\ C\ \{R\} \qquad \mathsf{fp}(C) \cap \mathsf{fv}(R') = \emptyset}{\{P \ast R'\}\ C\ \{R \ast R'\}}\ \ \text{(Frame)}
```

Falha de compensação. Se uma compensação falha, ou se as sondas do ARX não observam R, o bloco executa o efeito `CompensationFailed(residual)`, em que residual lista os efeitos cujo R não foi estabelecido. O efeito está na linha do `atomic`; agente que não o trata na sua fronteira não compila (E0503).

Irrevogáveis. Continuam proibidos dentro de `atomic` e só aparecem via `commit`, depois que a fase compensável se confirmou. A cláusula `observed by` declara a sonda que confirma o estado esperado; falha da sonda leva a escalonamento, nunca a compensação.

Pós-condições em posição de controle. O `await` de um `atomic` exige integridade ⊒ Endorsed e cobertura: o predicado deve quantificar sobre um superconjunto do fecho de footprints fp* dos efeitos executados, que a linha de efeitos carrega simbolicamente (`Cluster.Write[fp]`). Verificar um deployment escolhido pelo alerta enquanto se age sobre outro é erro de compilação (E0417).

Isolamento. A semântica é de saga, não de isolamento. Quando isolamento importa, `atomic(…) lease fp` adquire leases sobre o footprint via `Shared[T, linear]`, e `entry` denota o estado abstrato no início do bloco sob lease. `speculate { a } or { b }` segue como antes: ramos em sandboxes que só admitem efeitos reversíveis.

3.5 Efeitos remotos não instantâneos

Um timeout diz que o agente parou de esperar, não que o mundo deixou de mudar. Aletheia tipa essa diferença.

Chaves de idempotência lineares. Toda chamada remota com consequência recebe um `own IdemKey`, cunhado pelo ARX com a capacidade `Cap[IdemKey.Mint]` e registrado no journal. Com multiplicidade 1, a chave não pode ser reutilizada em outra operação nem descartada. Repetições da mesma operação lógica reusam a mesma chave.

TimeoutUnknown. Quando o prazo expira, a chamada não devolve falha: executa o efeito `TimeoutUnknown(p)`, com `p : Pending[k]`, também linear. O token precisa ser consumido exatamente uma vez — por uma reconciliação que chegue a `Done` ou `Failed`, por `transfer` ao reconciliador durável ou por `handoff` a uma fila humana. Descartá-lo é erro de multiplicidade (E1107).

```
\dfrac{\text{resposta antes do prazo}}{\langle \mathsf{call}\ t(c, a, k),\ J
\rangle \longrightarrow \langle \mathsf{Done}(b),\ J[k \mapsto b] \rangle}
\qquad \dfrac{\text{prazo expirado}}{\langle \mathsf{call}\ t(c, a, k),\ J
\rangle \longrightarrow \langle \mathsf{perform}\ \mathsf{TimeoutUnknown}
(\mathsf{Pending}\ k),\ J[k \mapsto ?] \rangle}
```

Protocolo de reconciliação. A reconciliação é um tipo de sessão entre o agente e o provedor, limitado por combustível e backoff. O esgotamento devolve o token a quem chamou como `Exhausted(p)`; nunca o descarta.

```
protocol Reconcile[k](Agent, Provider) {
  rec loop {
    Agent -> Provider : Status(k)
    choice at Provider {
      Done:    Provider -> Agent : Receipt;      end
      Failed:  Provider -> Agent : Reason;       end
      Unknown: Provider -> Agent : Retry(after); continue loop
    }
  }
}
```

Classe de provedor	Como reconciliar	Pode entrar em `atomic`
`idempotent`	reenviar com a mesma chave	sim
`queryable`	consultar o status pela chave	sim
`opaque`	impossível	não; só via `commit`, com handler que escala

Recuperação após queda. Ao reiniciar, o ARX percorre o journal e devolve ao reconciliador cada entrada `k ↦ ?`. Teorema-alvo T9: toda chave cunhada termina em `Done`, `Failed` ou `Escalated`; nenhuma execução termina com resultado em aberto.

Interação com sagas. Uma compensação roda mesmo quando o efeito original está em aberto, porque C é total sobre P ∨ Q (3.4). O agente não precisa saber se a ação aconteceu para levar o mundo ao invariante de recuperação.

3.6 Execução durável e cadeia de custódia

- Toda fonte de não determinismo — amostras, respostas de ferramentas, leituras de relógio, ordem de mensagens — é efeito, e todo efeito vai para o journal append-only do ARX.
- Isso dá replay exato para depuração, retomada após queda a partir do último checkpoint e uma **cadeia de custódia** para cada efeito irrevogável: qual prompt, qual hash de modelo, qual política endossou, qual capacidade autorizou.
- Como `infer` é um efeito conhecido, o ARX agrupa chamadas de muitos agentes em batching contínuo e compartilha prefixos. Orquestração em Python não consegue fazer isso porque só enxerga chamadas HTTP opacas.

3.7 Mitigação de alucinação no runtime

1. **Propor–verificar.** O modelo propõe; um verificador decidível ou o kernel de prova decide. É o único caminho de uma saída neural até um tipo refinado com risco zero.
2. **Afirmação vs. observação.** Ações irrevogáveis exigem precondições `Obs`; afirmações do modelo precisam ser confirmadas por ferramenta.
3. **Proveniência de contexto.** Fatos vindos de resumos são rebaixados a `Claim` automaticamente.
4. **Consenso calibrado.** `quorum(k of n, models = [...])` devolve a resposta e a taxa de concordância. Amostras do mesmo modelo são correlacionadas, então concordância só vira certeza com certificado conformal sobre esse escore, nunca por hipótese de independência.
5. **Recuperação transacional.** Uma pós-condição observada que falha dispara compensações que levam o mundo a um estado que satisfaz o invariante de recuperação, confirmado por sondas (3.4); o passado não é desfeito.

4. Sintaxe canônica e exemplo prático

O exemplo abaixo é um agente de remediação de incidentes em Kubernetes, em cerca de 130 linhas. Ele lê logs de terceiros, usa um LLM para propor ações, impede qualquer ação fora da política, prova o orçamento sob contrato antes de rodar, compensa a fase reversível se o serviço não se recuperar e nunca trata um resultado remoto desconhecido como falha.

4.1 Gramática essencial

```
decl     ::= fn | type | model | tool | policy | cert | agent | protocol |
handler
tool     ::= "tool" ident "(" params ")" "->" type effects contract* class
provider latency
class    ::= "reversible"
           | "compensable" "by" expr "restoring" pred ("commutes" "on" expr)?
           | "irrevocable" ("observed" "by" pred)?
provider ::= "provider" ("idempotent" | "queryable" | "opaque")
bound    ::= ("tok" | "usd" | "wall" | "joule" | "bytes") "<=" quantity
           | "risk" "<=" "(" prob "," prob ")"
type     ::= … | type "~" "(" prob "," prob ")" "_" ident | type "^"
integrity
           | type "@" place | "own" type | "&" type
expr     ::= …
           | ident "<~" expr                                  amostragem
           | "infer" ident "{" fields "}"                     nó de
```

```
inferência
           | "refine" expr "by" expr ("retry" nat)? ("else" expr)?
           | "certify" expr "by" expr
           | "atomic" "(" expr ")" ("lease" expr)? "recover" pred block
               ("on" "failure" "(" ident ")" block)?
               ("on" "compensation_failed" "(" ident ")" block)?
           | "commit" expr                                        efeito
irrevogável
           | "within" "[" bounds "]" block
           | "quote" "(" expr ")"                                 dado nunca
vira instrução
handler  ::= "handle" ident "(" params ")" block
```

4.2 Programa completo

```
module ops.remediate

use std.{Text, Nat, List, Set}
use std.infer.{Model, Dist}
use std.remote.{IdemKey, Pending, reconcile, expo}
use k8s.{Cluster, ClusterState, Deployment, Receipt, Alert, Outcome, W}
use pager.{Page, handoff}

-- ① Modelo como valor; preço e latência são contrato Δ, conferido a cada
chamada
model planner : Model[
  weights   = "blake3:9f2c71e0…",
  tokenizer = o200k,
  ctx       = 128k tok,
  decode    = { temperature = 0.2, max_out = derived },
  price     = { in = $3.00 / Mtok, out = $12.00 / Mtok },
  latency   = { ttft <= 1.5 s, per_tok <= 20 ms },
]

-- ② A classe do efeito está na gramática do domínio
type Replicas = { n: Nat | 2 <= n && n <= 64 }

type Fix =                                          -- compensáveis
  | Scale    { deploy: Deployment, to: Replicas }
  | Rollback { deploy: Deployment, revision: Nat }

type Kick =                                         -- irrevogáveis
  | Restart  { deploy: Deployment }

type Diagnosis = {
  cause : Text[<= 400 tok],
```

```
  fixes : List[Fix, 0..3],
  kicks : List[Kick, 0..1],
} where fixes.len + kicks.len >= 1

-- ③ Contratos sobre o estado abstrato W; revert é atribuição, total em P v
Q
tool apply(c: Cap[Cluster.Write], f: Fix ^ Endorsed, k: own IdemKey) ->
Receipt
  ! {Cluster.Write[f.deploy]}
  ensures   W.spec(f.deploy) == f.target()
  compensable by revert(c, f.deploy, old(W.spec(f.deploy)))
    restoring W.spec(f.deploy) == old(W.spec(f.deploy))
  commutes on f.deploy
  provider queryable  latency <= 1 s

tool restart(c: Cap[Cluster.Write], r: Kick ^ Endorsed, k: own IdemKey) ->
Receipt
  ! {Cluster.Write[r.deploy]}
  irrevocable observed by W.ready(r.deploy) >= 1
  provider idempotent  latency <= 1 s

tool page(c: Cap[Page], msg: Text[<= 200 tok], k: own IdemKey) -> ()
  ! {Page}  irrevocable  provider idempotent  latency <= 1 s

-- ④ Política: pura, total e decidível
policy SafeRemediation(s: ClusterState)(d: Diagnosis) -> Bool =
     all_distinct(d.fixes.map(_.deploy) ++ d.kicks.map(_.deploy))
  && forall f in d.fixes:
       f.deploy in s.managed() && match f {
         Scale    { deploy, to }       => to.n >= s.min_healthy(deploy),
         Rollback { deploy, revision } => revision in s.history(deploy),
       }
  && forall r in d.kicks:
       r.deploy in s.managed() && s.ready(r.deploy) > s.min_healthy(r.deploy)

fn escalate(c: Cap[Page], keys: Cap[IdemKey.Mint], a: Alert ^ Untrusted,
            why: Text[<= 40 tok]) -> Outcome ! {Page} {
  commit page(c, "{why} · alerta: {a.title.head(120 tok)}", keys.mint())
  Outcome.Escalated
}

-- ⑤ Orçamento: contrato estático sob Δ; preempção e reconciliação no ARX
agent Remediator(
  cluster : Cap[Cluster.Read + Cluster.Write],
  oncall  : Cap[Page],
  keys    : Cap[IdemKey.Mint],
```

```
) budget [tok <= 60k, usd <= 0.50, wall <= 120 s, risk <= (0, 0)]
{
  on alert(a: Alert ^ Untrusted) -> Outcome
     ! {Infer, Cluster, Page, TimeoutUnknown, CompensationFailed,
ContractBreach} {

    let s    = cluster.snapshot()                        -- Obs ^ Trusted
    let logs = cluster.logs(a.deploy, last = 10 min)     -- Obs ^ Untrusted
                |> fit(8k tok, keep = errors_first)

    -- ⑥ Proposta neural, verificação decidível
    let draft = infer planner {
      system = "Você é SRE. Diagnostique e proponha ações.",
      state  = s.summary(1k tok),
      data   = quote(logs),
    }
    let plan = refine draft by SafeRemediation(s) retry 3
               else return escalate(oncall, keys, a, "nenhum plano seguro")

    -- ⑦ Fecho afetado: derivado só de plan (Endorsed) e s (Trusted), nunca
do alerta
    let affected = s.dependents_closure(
                     plan.fixes.map(_.deploy) ++ plan.kicks.map(_.deploy))

    -- ⑧ Fase compensável: lease, invariante de recuperação, pós-condição
sobre o fecho
    atomic(cluster) lease affected
      recover forall d in affected: W.spec(d) == entry.spec(d)
    {
      for f in plan.fixes { apply(cluster, f, keys.mint()) }
      await forall d in affected: cluster.slo(d).healthy() within 40 s
    } on failure(e) {
      commit page(oncall, "Compensado: {e.brief(150 tok)}", keys.mint())
      return Outcome.Compensated(e)
    } on compensation_failed(cf) {
      commit page(oncall, "INTERVENÇÃO: {cf.residual.brief(150 tok)}",
keys.mint())
      return Outcome.NeedsHuman(cf)
    }

    -- ⑨ Fase irrevogável: só depois de a fase compensável se confirmar
    for r in plan.kicks {
      commit restart(cluster, r, keys.mint())
      await forall d in affected: cluster.slo(d).healthy() within 30 s
        else return escalate(oncall, keys, a, "restart sem recuperação")
    }
```

```
    commit page(oncall, "Resolvido: {plan.cause.head(150 tok)}", keys.mint())
    Outcome.Resolved(plan)
  }

  -- ⑩ Resultado desconhecido nunca é tratado como falha nula
  handle timeout_unknown(p: Pending[_]) {
    match reconcile(p, fuel = 3, backoff = expo(2 s)) {
      Done(r)      => resume Done(r),
      Failed(e)    => resume Failed(e),
      Exhausted(q) => { transfer q to arx.reconciler; abort
Outcome.Pending(q.key) },
    }
  }

  handle contract_breach(b) { fail(b) }   -- dentro de atomic, a falha
dispara compensações
}
```

O que o leitor deve notar:

- **② e ④ definem o raio de explosão.** Não existe `Delete` no tipo, e qualquer ação fora de `s.managed()` é rejeitada. Restart pertence a `Kick`, então não pode ser escrito como compensável.
- **⑥ é a fronteira neuro-simbólica.** A saída neural só atravessa para um tipo refinado pelo verificador; o rótulo Endorsed é o que permite chamar `apply` e `restart`.
- **⑦ fecha a vulnerabilidade da v0.1.** O fecho afetado vem do plano endossado e do snapshot confiável; usar `a.deploy` na pós-condição é o erro E0417.
- **⑧ declara o que a recuperação garante.** As compensações restauram a especificação de cada deployment, comutam entre si e rodam em paralelo; falha de compensação tem tratamento obrigatório.
- **⑨ separa o irrevogável.** Restart só acontece depois da fase compensável; se a sonda falhar, o agente escala, sem fingir que desfaz.
- **⑩ consome todo resultado em aberto.** O token `Pending` é reconciliado, transferido ao reconciliador durável ou entregue a um humano. Um `abort` dentro de `atomic` dispara as compensações, que são totais mesmo com o resultado em aberto.

4.3 O compilador antes de rodar

Com as latências declaradas no módulo `k8s` (snapshot ⩽ 2 s, logs ⩽ 2 s, revert ⩽ 1 s), `athc` verifica os oito estratos:

```
$ athc check ops/remediate.ath
✓ E0 estrutura       46 definições, 0 buracos; hashes de gerador conferidos
✓ E1 multiplicidade  6 pontos de cunhagem; cada IdemKey e cada Pending
consumidos uma vez
✓ E2 efeitos         TimeoutUnknown, CompensationFailed, ContractBreach
tratados; linha externa vazia
✓ E3 integridade     apply, restart ⊒ Endorsed(SafeRemediation); pós-
condições ⊒ Endorsed
✓ E4 índices         List[Fix, 0..3], List[Kick, 0..1]; sem tensores neste
módulo
✓ E5 refinamentos    31 obrigações: 22 do cache do Codex, 9 novas (0,19 s)
                     recover: R(apply) por deployment, footprints disjuntos ⇒
invariante
                     cobertura: pós-condições ⊇ fp*(plan) = affected
✓ E6 orçamento       sob Δ, pior caminho: 29,2k tok · US$ 0,102 · 114,7 s
                     ⩽ [60k tok · US$ 0,50 · 120 s]
                     fora de Δ: reconciliação ⩽ 17 s por chave, coberta por
preempção ou transferência
✓ E7 risco           (0, 0): nenhum certificado estatístico no caminho
```

O pior caminho sob Δ soma 2 (snapshot) + 2 (logs) + 3 × 11,9 (inferência: 1,5 s + 520 tok × 20 ms) + 3 (apply) + 40 (pós-condição) + 1 (restart) + 30 (sonda do restart) + 1 (page) = 114,7 s. No caminho de falha, as três compensações comutam e rodam em paralelo: 1 s, não 3.

Três erros que o compilador pega em edições comuns. Primeiro, a pós-condição volta a usar o deployment do alerta:

```
error[E0417]: pós-condição de atomic com integridade Untrusted
   --> ops/remediate.ath:101:37
    |
101 |       await cluster.slo(a.deploy).healthy() within 40 s
    |                         ^^^^^^^^ derivado de `a: Alert ^ Untrusted`
    |
    = exigido: pós-condição ⊒ Endorsed, cobrindo fp* = affected
    = o bloco escreve em fp*(plan.fixes); `a.deploy` pode não pertencer a ele
    = ajuda: quantifique sobre `affected`
```

Segundo, alguém move o restart para dentro da transação:

```
error[T0302]: efeito irrevogável dentro de atomic
   --> ops/remediate.ath:100:29
    |
100 |       for r in plan.kicks { restart(cluster, r, keys.mint()) }
    |                             ^^^^^^^ restart: irrevocable
    |
    = ajuda: mova para depois do bloco, com `commit`
```

Terceiro, o handler esquece o token pendente:

```
error[E1107]: valor linear descartado: Pending[k]
   --> ops/remediate.ath:126:23
    |
126 |       Exhausted(q) => abort Outcome.Pending(q.key),
    |                 ^ `q` não é consumido em todos os caminhos
    |
    = consumidores válidos: reconcile, transfer, handoff
```

Remover o `on compensation_failed` produz E0503: efeito não tratado na fronteira do agente.

4.4 Garantia estocástica explícita

Quando a propriedade não é decidível, o certificado entra no tipo, amarrado ao gerador, e o risco entra no orçamento:

```
cert faithful : Cert[
  prop    = faithful,                        -- φ(fonte, resumo), definida
pelos rótulos de D
  gen     = #writer.postmortem_v3,           -- hash do fecho gerador
  score   = entailment(nli_judge#b71e),      -- o escore também tem hash
  calib   = "blake3:41aa07…",                -- D: 2 000 pares rotulados,
amostrados da produção
  bound   = (alpha = 0.02, delta = 1e-3),    -- controle de risco condicional
a D
  monitor = { lambda = 1e-3, audit_every = 500 },
]

fn postmortem(inc: Incident ^ Trusted)
  -> Option[{t: Text[<= 300 tok] | faithful(inc.timeline, t)} ~ (0.02, 1e-
3)_D]
  ! {Infer writer, Cert faithful}  cost [risk <= (0.02, 1e-3)]
{
  let t <~ infer writer.postmortem_v3 { timeline = inc.timeline }
  certify t by faithful(source = inc.timeline)   -- None se o escore exceder
o limiar ou o monitor bloquear
}
```

Um agente que chame `postmortem` duas vezes acumula risco (0,04, 1e-3): α soma, δ não, porque o certificado é o mesmo. Com `risk <= (0.03, 1e-3)`, o agente não compila. Os dois erros novos da v0.2:

```
error[E0421]: certificado estatístico sobre fonte não confiável
   --> ops/support.ath:18:3
    |
 18 |   certify t by faithful(source = ticket.body)
    |                                  ^^^^^^^^^^ Text ^ Untrusted, sem
modelo de ameaça
    |
    = permutabilidade não vale sob adversário; (α, δ) seria vazio
    = ajuda: use `refine … by <policy>`, ou restrinja a entrada a Bounded(ρ,
ε, x₀)
             com um RobustCert

error[E0430]: certificado incompatível com o gerador
   --> ops/postmortem.ath:14:3
    |
    = faithful foi calibrado para #writer.postmortem_v3 (blake3:77c1…)
    = o gerador agora é #writer.postmortem_v4 (blake3:e09a…): template
alterado
    = ajuda: athc certify faithful --gen writer.postmortem_v4
```

4.5 O que cada construção substitui

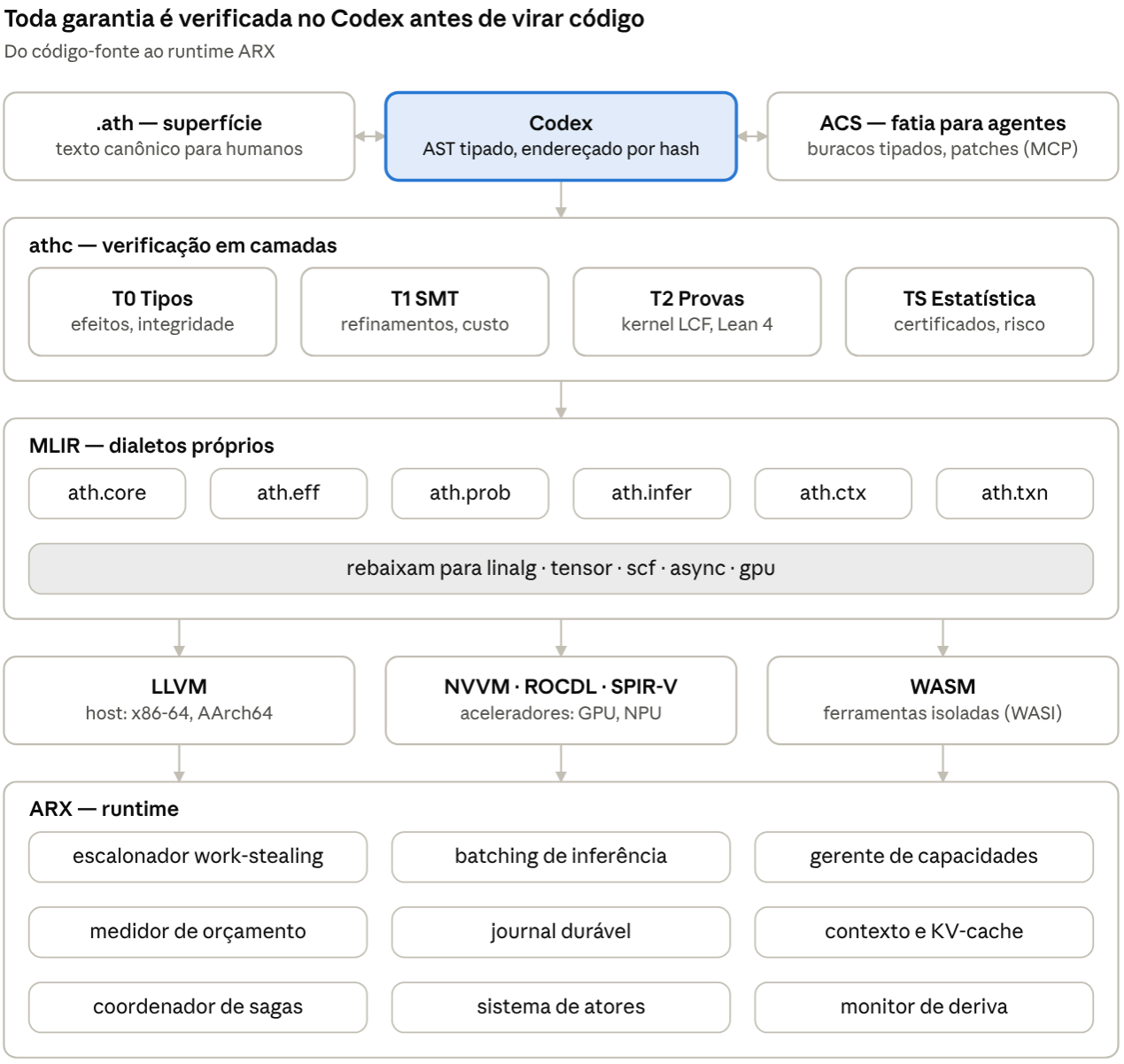
Construção Aletheia	Equivalente em Python hoje	Garantia ganha
`type Diagnosis` + `infer`	esquema Pydantic, parser, laço de reparo de JSON	saída válida por construção
`fit(8k tok)`, `Text[<= n tok]`	tiktoken + truncamento manual	estouro de contexto é erro de compilação
`budget […]` sob Δ	contadores de custo, `asyncio.wait_for`	limite provado sob contrato; violação de contrato sinalizada
`Cap[…]`	chaves de API globais, checagens ad hoc	efeito sem capacidade é inexprimível
`^ Untrusted` + `refine … by policy`	separação manual de prompts	injeção não escolhe ação fora da política
`atomic … recover`	saga com try/except e pilha de desfazer	estado de recuperação provado e observado
`on compensation_failed`	exceção engolida	falha de compensação nunca passa calada
`own IdemKey` + `Pending`	chaves de idempotência por convenção	nenhum resultado remoto desconhecido é esquecido
`commit`	convenção de equipe	irrevogável nunca antes da validação
`certify` + `risk (α, δ)`	—	risco estatístico somado, amarrado ao gerador, proibido sob adversário
journal do ARX	logging e tracing	replay exato e cadeia de custódia

Nossa estimativa é que o mesmo agente em Python exija de 400 a 600 linhas espalhadas por seis a oito bibliotecas, sem nenhuma dessas garantias verificada estaticamente.

5. Bootstrap, interoperabilidade e roteiro

Aletheia é bootstrapada em Rust sobre MLIR e LLVM, e sua primeira versão compila para Python. Assim as garantias estáticas valem desde o primeiro dia, sem exigir que ninguém abandone a infraestrutura que já roda.

5.1 Pilha de implementação



pipeline · do código-fonte ao runtime ARX

O Codex é a única fonte; texto e fatia ACS são vistas dele. Na fase 1, um emissor direto gera Python + asyncio a partir do núcleo verificado, antes de o pipeline MLIR existir.

- **Frontend (Rust).** Parser incremental e tolerante a erros; Codex como DAG de Merkle com BLAKE3; verificador bidirecional incremental no estilo salsa / rust-analyzer; servidor LSP para humanos e servidor MCP para agentes (fatias ACS, patches, diagnósticos estruturados).
- **Solvers e provas.** Z3 e cvc5 via SMT-LIB; kernel de provas pequeno no estilo LCF, escrito em Rust; exportação de obrigações para Lean 4 quando a prova exige matemática pesada.
- **Middle-end (MLIR).** Dialetos próprios rebaixam para os dialetos upstream:

Dialeto	Representa	Rebaixa para
`ath.core`	λ-cálculo tipado em SSA	`func`, `scf`, `arith`
`ath.eff`	efeitos e handlers	passagem de evidência (estilo Koka), CPS seletivo
`ath.prob`	`sample`, `observe`, handlers de inferência	`ath.core` + chamadas ao ARX
`ath.infer`	nós de inferência, grafos de prompt, plano de prefixos	chamadas ARX ao servidor de inferência
`ath.ctx`	segmentos, compactação, KV	runtime ARX
`ath.txn`	`atomic`, `commit`, compensações	coordenador de sagas do ARX
tensores	`einsum`, formas com eixos nomeados	`linalg`, `tensor`, `gpu` → NVVM, ROCDL, SPIR-V

- **Backends.** LLVM para host; NVVM, ROCDL e SPIR-V para aceleradores; Python + asyncio na fase 1; componentes WASM para ferramentas em sandbox.
- **Runtime ARX (Rust).** Escalonador work-stealing para CPU e streams de acelerador; batcher de inferência; gerenciador de capacidades; medidor de orçamento com preempção; journal durável; gerenciador de contexto e KV; coordenador de sagas; sistema de atores; monitor de deriva de certificados.
- **Por que Rust como hospedeiro.** A base confiável do compilador e do runtime precisa de segurança de memória sem GC, há bindings maduros para MLIR, e o ecossistema de compilação incremental já existe. O frontend passa a ser escrito em Aletheia na fase 4.

5.2 Interoperabilidade

A regra de fronteira é sempre a mesma: o que vem de fora entra com o tipo mais conservador e é estreitado por anotação explícita.

Ecossistema	Mecanismo	Tipo na fronteira
Python	`extern py "numpy" as np`	`Dyn ^ Untrusted ! {Py}`, com contratos e atribuição de culpa; stubs `.athi` estreitam gradualmente
PyTorch / JAX	DLPack, sem cópia	`Tensor[d, S] @dev`, forma verificada na fronteira

Ecossistema	Mecanismo	Tipo na fronteira
Pesos	safetensors, GGUF, ONNX	`Model[…]` identificado por hash
C / C++ / Rust	ABI C via LLVM	`extern` exige `assume`, registrado na auditoria
MCP	`import mcp "servidor"` gera `tool` a partir do JSON Schema	`irrevocable` e saída `Untrusted` até anotação humana
APIs de modelos	provedor declarado em `model` (compatível com OpenAI, Anthropic, vLLM, SGLang)	custo e latência no κ do modelo
Sandbox	componentes WASM com WASI baseado em capacidades	capacidades mapeadas uma a uma

5.3 Roteiro



roteiro · 5 fases, 4 portões, 36 meses

Os portões usam as métricas da seção 5.4; G1 é o teste decisivo da tese, porque mede injecção, edição por agentes e orçamento em produção real.

5.4 Métricas que decidem se a tese se sustenta

Cada fase só avança se a métrica correspondente for medida contra um baseline em Python.

Métrica	Hipótese	Como medir
M1 Injeção de prompt	nenhuma ação fora da política executada sob ataque	AgentDojo, contra baseline Python e CaMeL

Métrica	Hipótese	Como medir
M2 Edição por agentes	mais patches aceitos na primeira tentativa do que em Python ou Rust	tarefas de edição com fatias ACS vs. arquivos inteiros
M3 Contexto	tokens por edição proporcionais à interface, não ao repositório	repositórios de 10 mil a 1 milhão de linhas
M4 Orçamento	zero estouros não tratados	journal do ARX em produção-piloto
M5 Desempenho	throughput de agentes concorrentes maior que o da orquestração em Python	mesma carga, com e sem batching e reuso de prefixo

Apêndice — teoremas, não-objetivos e referências

A.1 Teoremas-alvo do núcleo λ_Ath

O núcleo tem semântica operacional de passos pequenos e semântica denotacional numa mônada graduada indexada por (ε, β) sobre espaços quase-Borel. Cada teorema é entregável de uma fase do roteiro, com prova mecanizada em Lean 4.

Teorema	Enunciado	Estratégia de prova	Fase
T1 Segurança de tipos	programa bem tipado progride ou termina, e tipos são preservados	progresso + preservação	F0
T2 Confinamento de efeitos	todo efeito executado pertence a uma capacidade alcançável pelo agente	relação lógica de capacidades de objeto	F0
T3 Não-interferência com endosso robusto	execuções que diferem só em entradas Untrusted coincidem nos valores Trusted; valores Endorsed(P) satisfazem P em ambas	relação lógica binária	F1
T4 Contrato de orçamento	sob Δ honrado, custo ⩽ β; toda violação de Δ sinaliza `ContractBreach`; o ARX nunca inicia chamada cuja reserva excede o saldo	semântica instrumentada com custo e reservas	F1

Teorema	Enunciado	Estratégia de prova	Fase
T5 Correção de risco	com probabilidade ⩾ 1 − Σ δ_c sobre os conjuntos de calibração, o risco de falsa aceitação é ⩽ Σ α nos usos, para fontes Trusted ou Bounded dentro de ε	desigualdade de Boole sobre a semântica probabilística; pior caso na bola para Bounded	F1
T6 Recuperação de sagas	em falha de `atomic`, ou o invariante de recuperação é observado, ou `CompensationFailed` é levantado com o resíduo exato; nenhum efeito irrevogável ocorre dentro	lógica de separação com enquadramento sobre W; indução sobre o journal	F1
T7 Ausência de deadlock	agentes tipados contra projeções de um protocolo global bem formado não travam	tipos de sessão multipartes	F2
T8 Elaboração estratificada	correta e completa em relação ao sistema declarativo nos fragmentos declarados; E0–E3 com soluções principais	indução sobre os estratos; aciclicidade do grafo de consumo	F0
T9 Nenhum resultado perdido	toda `IdemKey` cunhada termina em `Done`, `Failed` ou `Escalated`	linearidade de `Pending` + recuperação pelo journal	F1

A.2 Não-objetivos

- **Provar correção semântica de pesos neurais.** Modelos são oráculos encerrados; toda garantia sobre suas saídas atravessa um verificador ou um certificado.
- **Substituir linguagens de kernel** como CUDA e Triton. Kernels escritos à mão entram como `extern kernel` com contrato de forma.
- **Ser assistente de provas geral.** A camada T2 serve a módulos críticos; matemática pesada vai para Lean 4.
- **Isolamento serializável sobre o mundo externo.** A atomicidade sobre `@world` é semântica (saga), por definição.

A.3 Referências

- Andreotti, B.; Lachnitt, H.; Barbosa, H. *Carcara: An Efficient Proof Checker and Elaborator for SMT Proofs in the Alethe Format*. TACAS 2023.
- Angelopoulos, A. N. et al. *Conformal Risk Control*. ICLR 2024.
- Angelopoulos, A. N. et al. *Learn then Test: Calibrating Predictive Algorithms to Achieve Risk Control*. arXiv, 2021.
- Atkey, R. *Syntax and Semantics of Quantitative Type Theory*. LICS 2018.
- Bates, S. et al. *Distribution-Free, Risk-Controlling Prediction Sets*. Journal of the ACM, 2021.
- Debenedetti, E. et al. *AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents*. NeurIPS 2024.
- Debenedetti, E. et al. *Defeating Prompt Injections by Design* (CaMeL). arXiv, 2025.
- Findler, R. B.; Felleisen, M. *Contracts for Higher-Order Functions*. ICFP 2002.
- Garcia-Molina, H.; Salem, K. *Sagas*. SIGMOD 1987.
- Gendler, A. et al. *Adversarially Robust Conformal Prediction*. ICLR 2022.
- Heunen, C.; Kammar, O.; Staton, S.; Yang, H. *A Convenient Category for Higher-Order Probability Theory*. LICS 2017.
- Honda, K.; Yoshida, N.; Carbone, M. *Multiparty Asynchronous Session Types*. POPL 2008.
- Jeary, L. et al. *Verifiably Robust Conformal Prediction*. NeurIPS 2024.
- Kaminski, B. L. et al. *Weakest Precondition Reasoning for Expected Run-Times of Probabilistic Programs*. ESOP 2016.
- Katsumata, S. *Parametric Effect Monads and Semantics of Effect Systems*. POPL 2014.
- Lattner, C. et al. *MLIR: Scaling Compiler Infrastructure for Domain Specific Computation*. CGO 2021.
- Leijen, D. *Type Directed Compilation of Row-Typed Algebraic Effects*. POPL 2017.
- Mokhov, A.; Mitchell, N.; Peyton Jones, S. *Build Systems à la Carte*. ICFP 2018.
- O'Hearn, P.; Reynolds, J.; Yang, H. *Local Reasoning about Programs that Alter Data Structures*. CSL 2001.
- Odersky, M.; Sulzmann, M.; Wehr, M. *Type Inference with Constrained Types*. TAPOS, 1999.
- Omar, C. et al. *Live Functional Programming with Typed Holes*. POPL 2019.
- Racordon, D. et al. *Implementation Strategies for Mutable Value Semantics*. Journal of Object Technology, 2022.
- Rehof, J.; Mogensen, T. *Tractable Constraints in Finite Semilattices*. Science of Computer Programming, 1999.

- Rondon, P.; Kawaguchi, M.; Jhala, R. *Liquid Types*. PLDI 2008.
- Siek, J.; Taha, W. *Gradual Typing for Functional Languages*. Scheme and Functional Programming Workshop, 2006.
- Vovk, V.; Nouretdinov, I.; Gammerman, A. *Testing Exchangeability On-Line*. ICML 2003.
- Xi, H.; Pfenning, F. *Dependent Types in Practical Programming*. POPL 1999.
- Zdancewic, S.; Myers, A. C. *Robust Declassification*. CSFW 2001.
- Zheng, L. et al. *SGLang: Efficient Execution of Structured Language Model Programs*. NeurIPS 2024.
- Unison — código endereçado por conteúdo.